

RESEARCH ARTICLE

Visual Explanation With Action Query Transformer in Deep Reinforcement Learning and Visual Feedback via Augmented Reality

HIDENORI ITAYA¹, (Graduate Student Member, IEEE), WANTAO YIN²,
TSUBASA HIRAKAWA³, (Member, IEEE), TAKAYOSHI YAMASHITA⁴, (Member, IEEE),
HIRONOBU FUJIYOSHI², (Member, IEEE), AND KOMEI SUGIURA⁵, (Member, IEEE)

¹Chubu University, Kasugai-shi, Aichi 487-8501, Japan

²Department of Robotics, Chubu University, Kasugai-shi, Aichi 487-8501, Japan

³Center for Mathematical Science and Artificial Intelligence, Chubu University, Kasugai-shi, Aichi 487-8501, Japan

⁴Department of Computer Science, Chubu University, Kasugai-shi, Aichi 487-8501, Japan

⁵Department of Computer Science, Keio University, Yokohama, Kanagawa 223-8522, Japan

Corresponding author: Hidenori Itaya (itaya@mprg.cs.chubu.ac.jp)

This work was supported by the New Energy and Industrial Technology Development Organization (NEDO) under Project JPNP20006.

ABSTRACT Deep Reinforcement Learning (DRL) agents possess powerful control capabilities and have potential applications in robotics and other fields. However, the closed box properties of DRL agent models still make it difficult to interpret their decision-making processes. One research area that addresses this challenge is eXplainable Reinforcement Learning (XRL). Conventional visual explanation methods in XRL focus only on the rationale behind a single selected action and are insufficient for a more comprehensive analysis of an agent's decision making. In addition, visualizing attention only as an image is problematic in real-world environments such as robotics because it does not clearly map attention to physical space, limiting user understanding. To overcome these limitations, we propose Action Q-Transformer (AQT), an XRL method that uses a transformer encoder-decoder architecture with action information as a query. In this way, AQT calculates explicit attentions for each action that an agent may select, resulting in a DRL agent model that is easier to interpret. Furthermore, we introduce a visual feedback method using augmented reality (AR) to project these attentions directly into the physical environment. Through experiments on the Atari 2600 video game strategy task and a robot control task in an indoor environment, we demonstrate that AQT can analyze agent decision making in detail. Also, user evaluation in a robot task confirms that AR-based visual feedback effectively improves the understanding of the agent's behavior.

INDEX TERMS Explainable AI, deep reinforcement learning, augmented reality, video games, robot control.

I. INTRODUCTION

Deep reinforcement learning is attracting interest as a technology that demonstrates high performance in various control tasks in unknown environments. In 2015, Deep Q-Network (DQN), which introduced function approximation using Deep Neural Network (DNN) to Q-learning [1], was proposed, and it outperformed professional human

players in the video game task [2]. Inspired by this research, Deep Reinforcement Learning (DRL), which combines deep learning and reinforcement learning, has attracted interest due to its ability to handle multi-dimensional state data such as images. In particular, a key characteristic of DRL is that the agent can accumulate experience through exploration of the environment and learn the optimal behavior of the agent in the environment. Unlike supervised learning, it does not require a large amount of labeled data to be prepared in advance, so it is highly flexible in adapting to

The associate editor coordinating the review of this manuscript and approving it for publication was Ikramullah Lali.

unknown environments. In addition, since the Transformer was proposed in the field of natural language processing in 2017 [3], the transformer architecture has demonstrated remarkable performance in various tasks in the supervised learning field [4], [5], [6]. The Transformer is superior to conventional Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) in its ability to capture long-term dependencies in time-series data. As a result, it is also expected to be effective in the DRL field, and many methods that apply the transformer structure to DRL have been proposed [7], [8], [9], [10]. Due to these technological developments, the performance of DRL agents has improved significantly, and it has become possible to learn optimal agent behavior even in environments with a vast state space. Such high-performance DRL agents are being applied in a variety of real-world settings, including video games [11], robotics [12], traffic signal control [13], autonomous driving [14], healthcare [15], and alignment of language models [16].

Although DRL achieves high performance in various control tasks, it uses DNN for agent decision-making, resulting in its internal processing becoming a closed box. This black-boxing directly affects the reliability of the agent model, and poses an obstacle to the application of DRL agents to real-world applications. A research field called eXplainable Reinforcement Learning (XRL) is attracting attention for this problem. XRL is a research field that aims to realize agent models with high explainability and interpretability in DRL. Most of the previous research on XRL has been focused on methods for interpreting the agent's decision-making through post-processing of the learned model, such as calculating saliency maps for the agent model. These methods tend to require changing the structure of the agent model after learning, and have very high computational costs, which is not desirable for agents operating in real time. For example, changing the agent model structure after learning leads to a decrease in the agent model's performance. Also, the extremely high computational cost can cause significant delays in real-time synchronization with the real environment. To overcome these issues, the focus is on XRL methods that can interpret the model structure itself in advance [17], [18].

Based on the above background, we propose Action Q-Transformer (AQT), a Q-learning-based DRL method that has high interpretability for agent decision-making. AQT uses a Transformer encoder-decoder structure to calculate Q-values that contribute to action selection in the decoder. Our decoder incorporates a mechanism to obtain attention for each action using action queries that express action information as queries. This enables the agent to obtain attention that is unique to each action, and to achieve detailed analysis of the agent's decision-making. In XRL, there have been many proposals for XRL methods that focus solely on the selected actions of the DRL model [17], [18], [19]. On the other hand, to the best of our knowledge, this is the first time that a DRL model has been analyzed in detail by

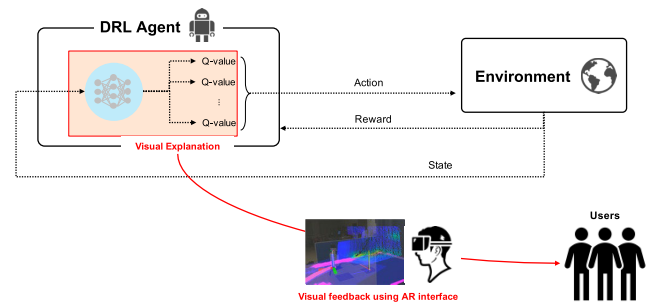


FIGURE 1. Overview of our research.

proposing an XRL method that focuses on all possible actions (reference fig. 1).

We also propose a system that uses Augmented Reality (AR) to provide visual feedback as a framework that enables users to intuitively and immediately understand the robot's behavior in robot control tasks where the user and robot interact in real time in a real environment. AR is a technology that overlays virtual visual information on the real world, and it can provide rich visual feedback without being constrained by physical properties. In our study, we use AR to visualize the attention map, which shows the robot's decision-making process in the real world, so that users can visually and easily understand the robot's decision-making process through the AR interface. In conventional visual feedback methods to users in XRL, it is standard to visualize saliency maps and attention maps for input information as images. However, these methods are not intuitive because they do not take depth information into account when understanding robots that operate in the real world, so they are not suitable for user understanding. On the other hand, to the best of our knowledge, this is the first time that a visual feedback method for users has been used to visualize attention in real world using an AR interface (reference fig. 1).

The three contributions of this paper:

- We propose a new Q-learning-based XRL method called Action Q-Transformer (AQT). AQT introduces a Transformer encoder-decoder structure that uses action information as a query to obtain attention that is unique to each action. This approach achieves a DRL model that offers high interpretability of the agent's decision-making.
- We propose a visual feedback method using Augmented Reality (AR), as a system allowing users to intuitively and immediately understand the agent's decision-making. We visualize the attention map, which shows the robot agent's decision-making, in the real world using AR. This approach allows users to easily understand the robot's decision-making in real-time robot control tasks through an AR interface.
- By introducing a transformer structure into the DRL agent model, we achieved control performance comparable to or exceeding the baseline in experiments on the Atari 2600 benchmark and autonomous robot navigation tasks.

II. RELATED WORKS

In this section, we describe the typical methods in Deep Reinforcement Learning (DRL), and explain the research on interpretability/explainability of agent models in DRL, which is most relevant to our research. We also explain the research on the application of Augmented Reality in robotics.

A. DEEP REINFORCEMENT LEARNING

1) Q-LEARNING-BASED DRL METHODS

The Deep Q-Network (DQN) delivers higher performance than previous methods on video game tasks [2]. DQN uses a neural network to represent the action value function (Q-value) $Q(s, a; \theta)$ and optimizes the network parameter θ to obtain the optimal action. Here, a is the agent's action, and s is the state observed from the environment. Since its introduction, many DRL methods based on Q-learning [1] have been developed.

Hasselt et al. proposed Double DQN (DDQN), which uses two Q functions to calculate Q-values, to solve the problem of overestimation in DQN [20]. Because DQN learns by randomly sampling acquired experience, it may also sample unimportant experiences. Prioritized Experience Replay (PER) achieves efficient learning by prioritizing important experience [21]. Wang et al. defined advantage, which represents the intrinsic value of the action, and proposed a Dueling network that decomposes the Q-value $Q(s, a; \theta)$ into state-value $V(s; \theta)$ and advantage $Adv(s, a; \theta)$ [22]. Here, θ is a parameter of the agent model. By estimating the state-value and advantage separately, even if the state-value is high, if the advantage is low, the action will not be selected. In other words, it achieves an accurate evaluation of actions independent of the state. The Q-value in DQN is expressed as an expected value of return. In contrast, Bellemare et al. proposed distributional reinforcement learning, in which the Q function is represented as a distribution [23]. This method improves the representation of Q-values and resolves unstable learning. The ϵ -greedy method *etc.* implemented in DQN cannot sufficiently explore the state space of the environment. Fortunato et al. proposed Noisy Networks, which add noise to the network that calculates actions, to achieve efficient exploration of the environment [24].

As described above, many methods based on DQN with various innovations have been published. Hessel et al. proposed Rainbow, which combines these methods and achieved a high score on the Atari 2600 [25]. Since then, several improved methods have been developed and are among the most important techniques in DRL [26], [27].

2) TRANSFORMER-BASED DRL METHODS

Transformer have shown impressive performance in various tasks in the supervised learning field, and applying the Transformer architecture has attracted interest in DRL as well.

In offline RL, where agents learn optimal policies from large-scale offline data, the RL problem can be treated as

a sequence modeling problem. For this reason, a number of methods have been proposed that use Transformer architecture, which offers superior performance for sequence information. Chen et al. proposed an offline RL method called Decision Transformer (DT) [7]. DT determines the next action by autoregressively predicting the past state/action/return sequence. However, DT has problems in considering long-term dependencies, and the learned policy may be biased toward maximizing short-term rewards. To overcome these challenges, new methods have been proposed. Kim et al. proposed the Decision Convformer, which enhances local time-series information by replacing the DT's self-attention with a convolutional layer [9]. This method prevents the agent from relying strongly on short-term information and enables it to learn a more stable policy. Also, Wang et al. proposed the Long-Short Decision Transformer (LSDT), which focuses on integrating information from both long and short time scales [8]. LSTD facilitates learning policies that maximize long-term returns by introducing a mechanism to enhance the capture of long-term dependencies. While these Transformer-based methods focus on offline RL, real-world applications require costly fine-tuning through task-specific interactions with the environment after pre-training. Zheng et al. proposed Online DT (ODT), in which the deterministic policy in DT is replaced by a probabilistic policy [10]. This method achieves high performance through fine-tuning by online-RL.

Other studies have attempted to obtain generalized policies from multiple large data sets. Multi-Game Decision Transformer (MGDT) is a method for learning generalized policies by integrating experiences in different game environments, within the DT framework [28]. MGDT processes different game play histories in a unified process and learns policy representations that can be adapted to multiple environments. Also, Gato enables consistent learning for various tasks by processing various input modalities (text, images, action data, *etc.*) in a unified format using a transformer structure [29]. Furthermore, the Decomposed Prompt Decision Transformer (DPDT) has acquired policies that perform effectively even on unknown tasks by prompt tuning [30]. This method extracts cross-task general knowledge by implementing a parameter-efficient multitask prompt tuning technique.

The transformer structure is reported to be effective in learning optimal policy for agent models. This strength has also been applied in model-based reinforcement learning and multi-agent reinforcement learning (MARL). Micheli et al. proposed Imagination with auto-Regression over an Inner Speech (IRIS), which predicts the next state's discrete token / reward / termination decision in an auto-regressive process by constructing a world model that introduces a transformer structure [31]. Additionally, based on IRIS, δ -IRIS [32], which predicts the difference between states, has been proposed and reported to improve the performance of the world model. Wen et al. proposed Multi-agent Transformer (MAT), which considers multi-agent decision making in a cooperative task as a sequence problem and

performs sequential decision making with a transformer model [33]. This method can use the high performance of the transformer for sequence information by sequential decision making, and obtains high-performance cooperative behavior in MARL. Hu et al. proposed CommFormer, which represents communication between agents as a learnable graph and introduces the transformer model into the graph modeling framework [34]. This method enables optimization of communication graphs through the global information processing of the transformer.

B. EXPLAINABLE REINFORCEMENT LEARNING

eXplainable AI (XAI) is gaining interest for its potential to improve the reliability of AI systems. As such, it is rapidly becoming an important research area for solving the closed box problem in AI. DRL is also showing potential for solving the problem of reliability in agent models. In the research area called eXplainable Reinforcement Learning (XRL), research focuses on analyzing the agent model's decision-making.

Greydanus et al. calculate perturbation images by applying a Gaussian filter to the gradient of back-propagation during the action selection process of an agent [19]. They analyze the agent's decision-making by obtaining saliency maps from these perturbation images. Weitkamp et al. proposed a bottom-up method for the actor to compute the policy by applying a Grad-CAM [35]-based gazing area calculation method [36]. However, these methods rely on back-propagation when analyzing agents' decision-making because they are bottom-up methods. Manchin et al. introduced self-attention to Proximal Policy Optimization [37], aiming at improving the score as well as analyzing the policy [38]. This method analyzes agents' decision-making by visualizing their attention to the policy. Zhang et al. proposed Attention-Guided Imitation Learning (AGIL), which guides the gazing area of a network based on a person's gaze information [39]. They estimated the human gaze by supervised learning and utilized it to augment the input to guide the agent's gaze during action selection. Rupprecht et al. focused on the importance of states that obtain significantly higher or lower rewards, and proposed a method for generating states of interest to trained DRL agents [40]. This method trains the generative model to output states that the agent recognizes from the real environment, and the model is then utilized to analyze the decision-making of the agent. Shi et al. proposed a self-supervised interpretable network (SSINet) that generates a fine-grained attention mask to highlight task-related information relevant to the agent's decision-making [41]. They generated an attention mask for the agent's decision-making by introducing SSINet before the agent's actor network. Mott et al. obtained two types of attention (context, spatial) by using query-based attention [17]. This method obtains context/spatial-related attention by generating an attention query from the query network using the LSTM state from the previous time step as input. In contrast,

our method explicitly obtains attention for each action by generating an action query using a one-hot vector of actions and then using it to calculate the advantage. Kenny et al. proposed the Prototype-Wrapper Network (PW-Net), which clarifies the inference process by introducing human-friendly prototypes into the network for action selection [18]. They showed that PW-Net does not degrade performance compared to the closed box model. In contrast, our method clarifies the inference process by using a Transformer encoder-decoder structure and introducing an action query. The introduction of the Transformer improves the scores in some game tasks compared to the baseline. Bekkemoen et al. proposed ASAP, an interpretable state abstraction method for clustering observed states using an attention map [42]. They report that state clustering using ASAP can appropriately classify semantically similar states and separate different states.

Other studies have reported XRL methods focusing on MARL. Mahjoub et al. proposed a measurement method for the contribution of each agent to the cooperative behavior in MARL [43]. This method utilizes difference rewards and linear computational cost to measure the same evaluation as the Shapley value in real-time. Also, Miskiw et al. combine DRL with agent-based modeling of simulated electricity markets to analyze agent behavior, focusing on modeling the bidding behavior of market participants [44]. This method utilizes clustering and DeepSHAP to analyze the factors behind agents' bidding behavior.

These XRL methods analyse agent decision-making by focusing only on the selected actions of the agent for the current state observed from the environment. In contrast, we consider that a framework that can analyse all possible actions is necessary to gain a detailed insight into the agent's decision-making. Our approach, the Action Q-Transformer (AQT), uses a Transformer encoder-decoder structure that uses action information as a query to obtain attention that is unique to each action. This enables visual explanations of all the actions that the agent can select.

C. AUGMENTED REALITY FOR ROBOTICS

With the advancement of robotics, there is a growing interest in the field of human-robot interaction, which focuses on the interaction between humans and robots. Conventional robot interaction often relies on small displays, and because of the limited screen size, it is difficult to present information related to the physical space outside the screen. However, there is potential to solve this problem by using an Augmented Reality (AR) interface [45]. AR is a technology that overlays virtual visual information on the real world, and it can provide rich visual feedback without physical constraints. In addition, by incorporating AR, visual feedback can be contained within the user's field of vision and is physically connected to the real world, so users can understand the robot's behavior more intuitively.

As an example of robotics using AR, Makhataeva et al. proposed an AR-based method to visually represent the hazardous areas around the robots [46]. This method

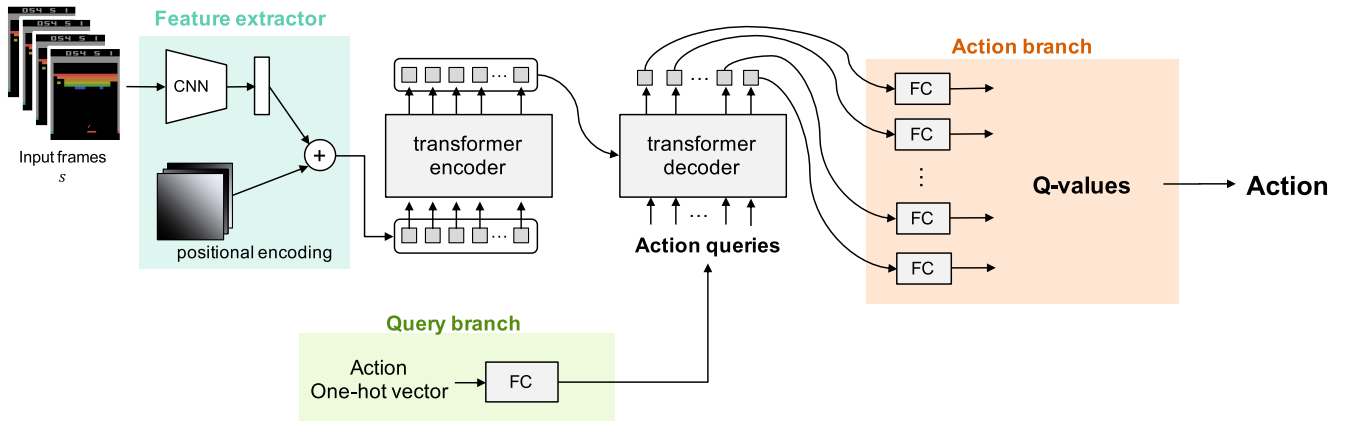


FIGURE 2. Overview of action Q-Transformer structure.

improves user safety and productivity by displaying the robot's surrounding environment using different colors to indicate varying levels of potential danger, allowing the user to avoid dangerous areas. Hedayati et al. proposed an AR interface for users to control drones [47]. This system shows users important data such as camera images from the drone's perspective and the drone's flight route via an AR device. This system enables users to control drones more intuitively, and it is expected to improve remote operability and reduce crashes. As these studies show, AR interfaces can provide users with expressive visual feedback, which can bring safety to users. We focus on the effectiveness of this AR interface, and achieve high interpretability of robot agents by providing visual feedback of the high-performance robot decision-making learned by DRL.

III. ACTION Q-TRANSFORMER

Deep reinforcement learning (DRL) agents have achieved high control performance in many studies, and are expected to be applied to various control tasks. However, the basis for the agent's decision-making process after learning is unclear, which hinders the application of DRL in real-world environments. Therefore, we propose an eXplainable RL (XRL) method, Action Q-Transformer (AQT), which introduces a transformer encoder-decoder structure into a DRL method based on Q-learning.

A. OVERVIEW OF AQT STRUCTURE

AQT consists of a Feature extractor, Transformer Encoder-Decoder, Query branch and Action branch, as shown in fig. 2. The encoder considers the relationship between the input image patches, and the Decoder considers the relationship between the encoder output and the action query that represents the action information. The action query represents the possible actions of the agent, and the action value function (Q-value) is calculated in the action branch using the corresponding query for each action. The action with the highest calculated Q-value is selected as the agent's action at that time.

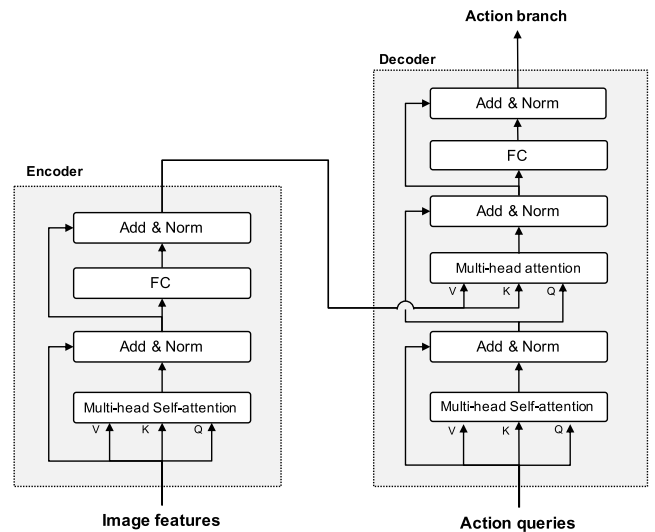


FIGURE 3. Details of transformer encoder-decoder.

B. FEATURE EXTRACTOR

The feature extractor obtains a feature map of a specific size from an input image using the Convolutional Neural Network (CNN). The acquired feature map is transformed into 1D features by adding location information to the feature map. Each element in the 1D features corresponds to each element in the feature map. In other words, each element of the feature map corresponds to a patch when the input image is divided into patches of the same size as the feature map. The acquired 1D features are used as input values to the Transformer Encoder.

C. TRANSFORMER ENCODER-DECODER

Our method implements a DRL model with high interpretability/explainability for agent behavior by introducing the Transformer Encoder-Decoder shown in fig. 3.

1) ENCODER-DECODER STRUCTURE

The encoder consists of a multi-head self-attention, a fully connected layer (FC), a residual mechanism, and

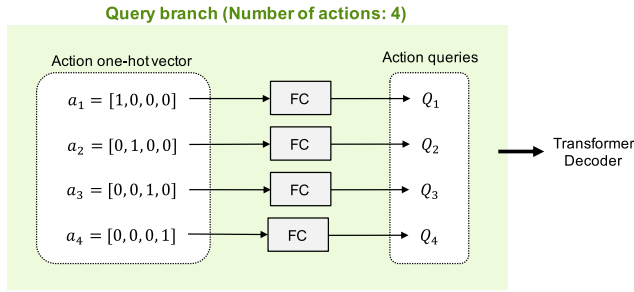


FIGURE 4. Details of query branch.

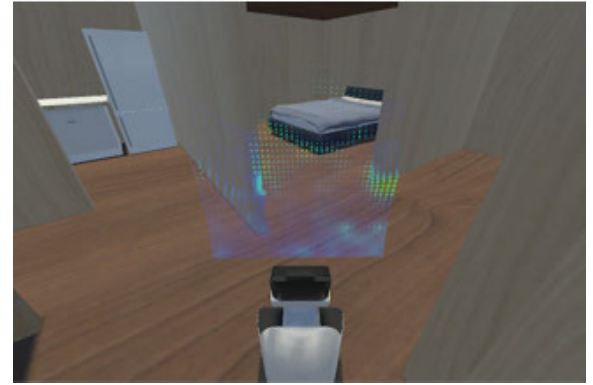
a normalization process (Add & Norm). Multi-head self-attention calculates the features by considering the relationship between elements in the feature map. The decoder consists of four modules: the same module as the encoder, adding the multi-head attention. The value and key of the multi-head attention are the features acquired by the encoder, and the query is the action queries acquired by the query branch (described below). The attention in the decoder's multi-head attention is calculated as

$$att_{de}(Q_{act}, F_{en}) = \text{softmax} \left(\frac{Q_{act} \cdot F_{en}^T}{\sqrt{\dim}} \right) F_{en}, \quad (1)$$

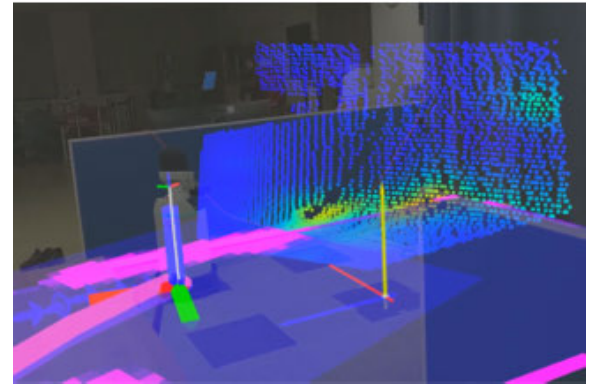
where Q_{act} is the action queries after multi-head self-attention, F_{en} is the encoder output value, $\dim$ is the dimensionality of Q_{act} and F_{en} , and $\text{softmax}(\cdot)$ is the softmax function. By using the decoder's output as input to the action branch, it acquires attention specific to each action from the decoder. By introducing a transformer encoder-decoder structure that uses this action query, it is possible to obtain attention to actions, which is an important element in understanding agent's decision-making, and to achieve a DRL agent model with high interpretability.

2) QUERY BRANCH

In our method, the calculation of the Q-value for each action is achieved by using the action information as a query for the transformer-decoder. Therefore, the action query used for the decoder is calculated in the query branch. Fig. 4 shows an example of the Query branch when there are four agent actions. The process flow in the query branch for this case is as follows. As our method employs a value-based DRL algorithm, we define a one-hot vector for each action, assuming that the agent's actions are discrete. Each one-hot vector is converted to an embedding vector at the FC layer and used as a query (action query) for the decoder. The number of queries is the same as the number of actions, and the FC layer parameters used to convert the queries to embedding vectors are updated during training, as in other layers. Thus, the Q-value of each action can be calculated using the action queries, which are learnable parameters.



(a) CG environment: visualizing attention using AR in simulation environment.



(b) Real world: visualizing attention using AR in real world.

FIGURE 5. Examples of attention visualization using AR.

D. ACTION BRANCH

The output of the decoder is calculated for each action query, i.e., a feature is calculated for each action. In the action branch, the Q-value is calculated from each action feature by the FC layer. The Q-values are calculated for the number of actions, and the action with the highest Q-value is selected as the agent's action. The structure of the action branch may be modified according to the value-based DRL algorithm employed.

E. TARGET TRAINED NETWORK

Since AQT employs a transformer encoder-decoder structure, it is considered to have the same properties as a transformer in learning. It is known that transformer performance has a scaling law that improves performance with the amount of training data [48], [49]. Therefore, the improvement of the AQT score is expected to depend on the number of training steps. We attempt to solve the above problem by proposing a loss function, Target Trained Q-network (TTQ), whose target value is the output of a trained baseline model. The baseline model is an existing value-based DRL algorithm that was used as the basis for constructing AQT.

The TTQ is designed as the squared error between the output Q_{aqt} of the AQT model and the target value, with the action value Q_{base} output by the learned baseline model as

the target value. The loss function L of AQT with TTQ L_{ttq} is calculated as

$$L = L_{aqt} + \alpha L_{ttq},$$

$$L_{ttq} = (Q_{base}(s_t, a; \theta_{base}) - Q_{aqt}(s_t, a; \theta_{aqt}))^2, \quad (2)$$

where L_{aqt} is the loss function similar to the loss function of the algorithm used for training (Rainbow in our method), α is the learning rate of L_{ttq} , and θ_{base} , θ_{aqt} are the network parameters of each model.

IV. VISUAL FEEDBACK METHOD FOR USERS USING AR INTERFACE

One of the most important factors in the interpretability/explainability of agent models in XRL is how to provide users with explanations and interpretations of the agent models. In general visual explanation methods, saliency maps and attention maps are often visualized as images overlaid on the input images. However, for tasks such as robot control that operate in the real world, visual feedback methods using images are not suitable for user understanding because 3D spatial information such as depth is difficult to understand and is not intuitive. Therefore, we focus on Augmented Reality (AR) interfaces. AR is a technology that overlays virtual visual information on the real world, providing expressive visual feedback without physical constraints. Using this advantage of AR, we propose an AR-based visual feedback method for decision-making of a high performance robot agent learned by DRL. The depth information obtained from the robot's depth sensor is converted into a set of points, and the points are colored according to the attention value. The attention used for visualization is the AQT decoder-attention for the action with the highest Q-value. An example of AR-based attention visualization is shown in fig. 5. fig. 5 (a) is an example of simulated visual feedback by AR in the CG environment, and fig. 5 (b) is an example of visual feedback by an AR device in the real world. The visualized attention is represented by a heatmap, where red indicates that the robot agent is strongly gazing at the object, and blue indicates that the robot agent is not gazing at the object. This visualization can be used to provide feedback of the robot agent's decision-making learned by DRL in a form that is easily understood for the user.

V. EXPERIMENTS

To evaluate the effectiveness of the Action Q-Transformer (AQT), we describe experiments using the OpenAI Gym video game strategy task [50] and a robot autonomous movement task in an indoor environment. In the robot autonomous movement task, we evaluate the performance of the AQT as well as the visual feedback to users using an Augmented Reality (AR) interface. The following sections describe and discuss each of the evaluation experiments and results.

A. VIDEO GAME STRATEGY TASK

In this section, we describe experiments using the OpenAI Gym video game strategy task. In this experiment, we used 50 Atari 2600 games and analyzed the agents' decision-making by visualizing the attention acquired in "Breakout" and "Seaquest". As the baseline method for AQT, we used the DRL algorithm Rainbow [25], which has a high score on Atari 2600. The compared methods are the baseline model "Rainbow" and our method "AQT". The end of training for each method is when the number of training steps reaches 5.0×10^7 . The episode ends either when a single playthrough of the game is completed or when the number of steps reaches 1.08×10^5 . The following two evaluation methods were used.

- Score comparison on the Atari 2600
- Visual explanations using attention

1) DETAILS OF VIDEO GAME AGENT MODEL

Since Rainbow employs a dueling network ([22]), it is necessary to separate the output into state-value and advantage for the Q-value calculation. Therefore, the action branch of the AQT model in this experiment consists of two branches: a value branch for calculating state-value and an advantage branch for calculating advantage value. The value branch uses the output value of the Encoder as the input value, while the Advantage branch uses the output value of the Decoder as the input value. This structure allows the encoder-attention to obtain the attention that contributes to the state-value and the decoder-attention to obtain the attention that contributes to the advantage. In the value branch and advantage branch, the state-value $V(s_t)$ and advantage value $Adv(s_t, a)$ are calculated for each branch, and the Q-value $Q(s_t, a)$ is obtained as the final output.

$$Q(s_t, a) = V(s_t) + Adv(s_t, a) - \frac{\sum_{i=1}^{n_a} Adv(s_t, a_i)}{n_a}, \quad (3)$$

where a is the agent's action, s is the state of the environment, t is the time, and n_a is the number of agent actions.

2) SCORE COMPARISON ON THE ATARI 2600

The mean scores for the 100 episodes on the Atari 2600 are shown in fig. 6. As we can see, AQT (gray) performs better than the baseline Rainbow (red dashed line) in games such as Bowling and Breakout. These are relatively simple games as they feature tasks with a low number of agent actions and a small environment search space. Therefore, we assume that AQT using the Transformer structure can accurately estimate the Q-value of each action and thereby improve the score. On the other hand, the AQT (gray) score improvement cannot be confirmed in Fishing derby and Seaquest *etc.*. Games for which no improvement in AQT scores could be confirmed are more difficult to learn than games for which scores improved. In such a task where learning is difficult, the Transformer scaling rule described in sec.III-E appears to have a significant impact. Hence, we consider that the AQT scores are lower than the baseline Rainbow scores in game tasks that are difficult to learn, such as Seaquest.

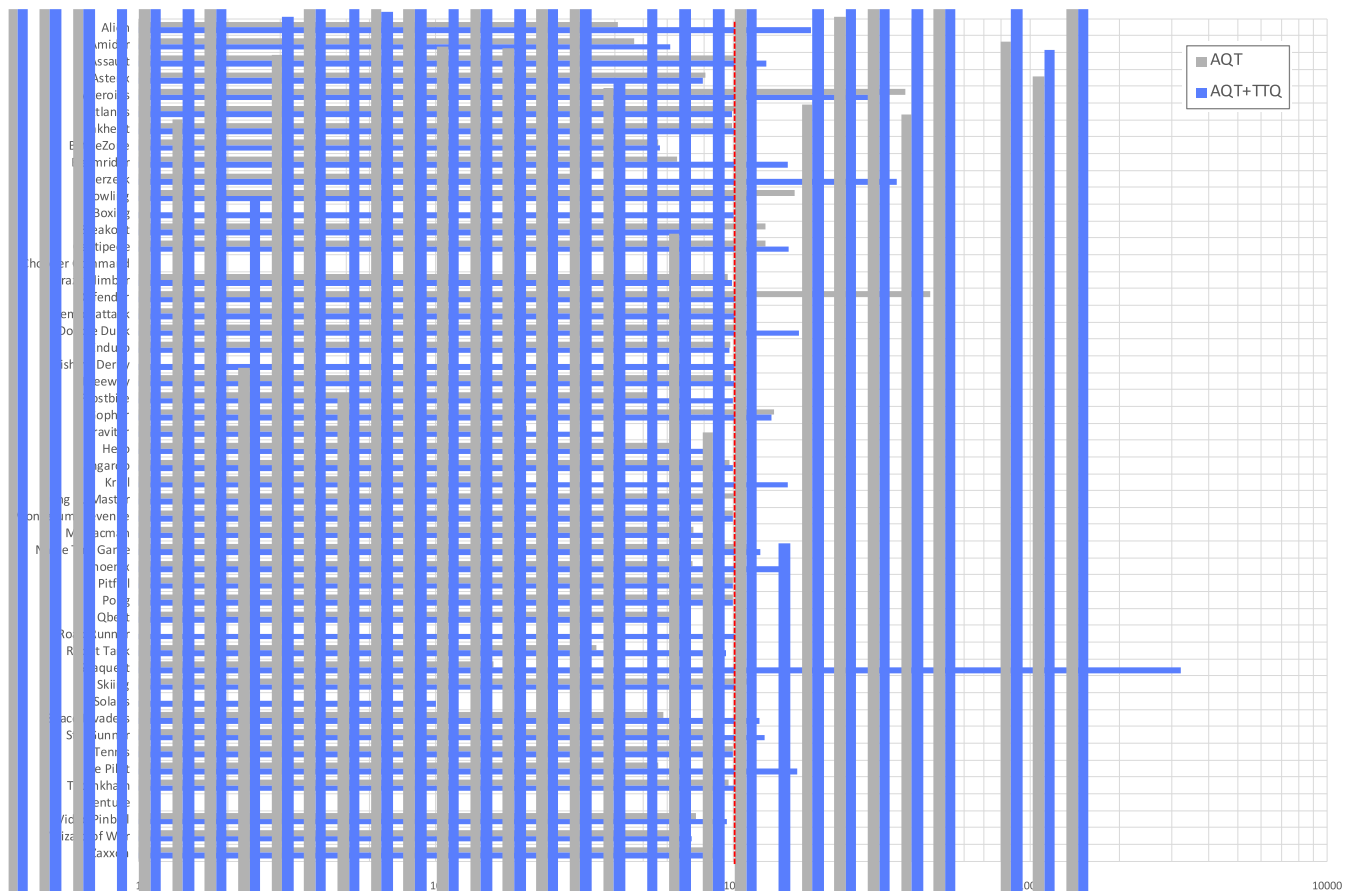


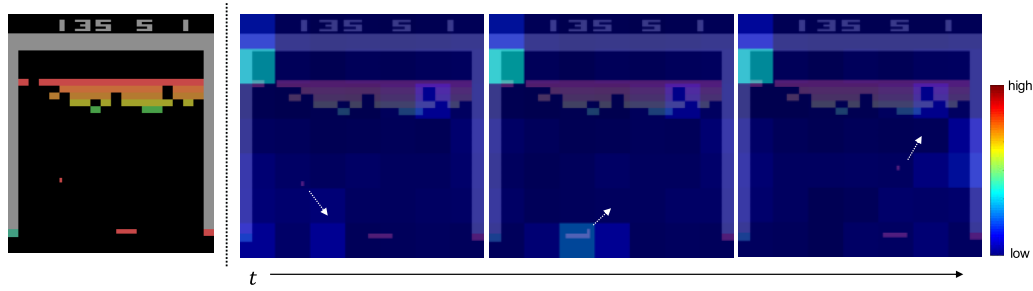
FIGURE 6. Mean scores over 100 episodes on Atari 2600. The graph shows the percentage of the average score of each method when the mean score of the baseline Rainbow model (red dashed line) is 100.

The AQT with TTQ (AQT+TTQ) in our experiments is implemented by linearly decreasing the learning rate α of TTQ from 1.0 to 0.0 over 2.5×10^7 steps in order to take into account the improvement in performance due to the loss function L_{aqt} of the AQT. In other words, the learning after 2.5×10^7 steps is the same as that for AQT. We can see from fig. 6 that AQT+TTQ improves its score compared to Rainbow in the tasks where AQT did not achieve high performance (Beamrider, Fishing Derby, Seaquest, *etc.*). On the other hand, AQT without the TTQ scored higher in games (Bowling, Breakout, *etc.*) in which the score improvement was confirmed in AQT. This is because the target Rainbow score was lower than the AQT score, and the TTQ may have prevented learning of AQT.

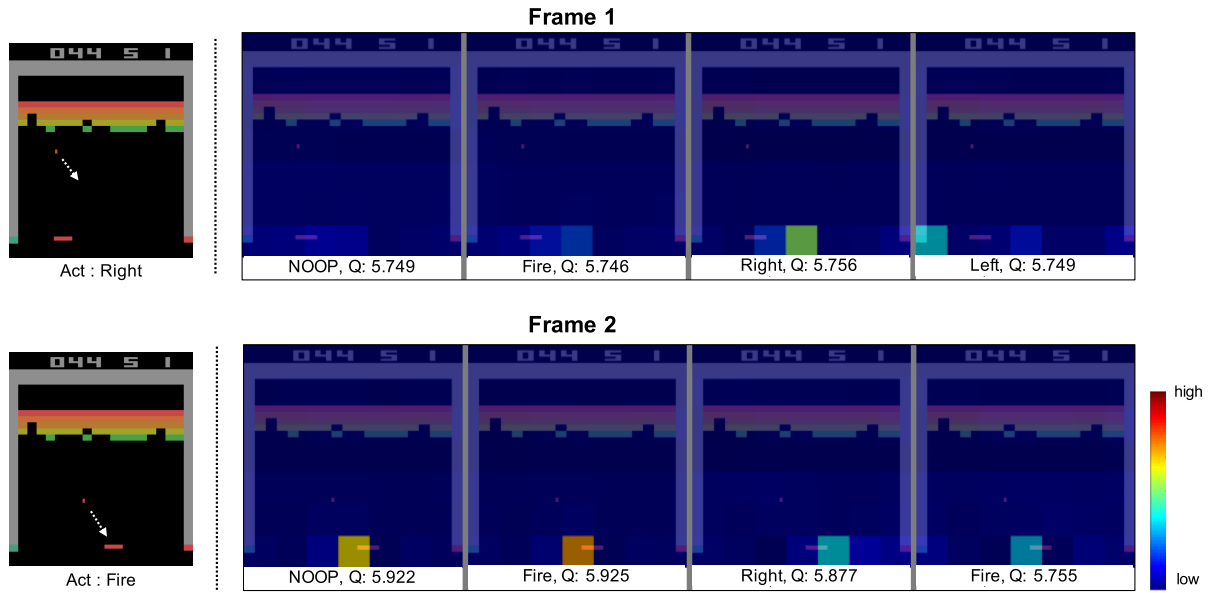
In games with a fixed viewpoint (the agent does not move from the screen, *etc.*), such as Asteroids and Gopher, the state space is simpler because there are relatively few changes on the game screen. In these games, both the AQT and AQT+TTQ scores are higher than the baseline. On the other hand, stage-type games such as Alien and Berzerk have a complex state space because the state changes significantly depending on the game stage transitions. In these games, the AQT scores are below the baseline. In particular, games

such as Chopper Command, Gravitar, Solaris, and Venture, in which the agent’s control objective differs from stage to stage, have more complex state spaces, and their AQT+TTQ scores are also below the baseline. These results show that for the same number of training steps, AQT scores lower than on tasks with complex state spaces than conventional DRL methods that do not use the transformer structure. This means that AQT requires more training steps than the DRL method without the transformer structure. This trend is consistent with transformer’s scaling rule. In other words, the AQT model has an advantage in environments that can take advantage of large-scale data and long-term dependencies, but may be disadvantageous to methods that do not use the transformer when the number of training steps is limited or when the environment changes rapidly and local adaptation is required.

These results show that our methods (AQT, AQT+TTQ) outperform the baseline model (Rainbow) on many video game tasks, and also confirm the effectiveness of the transformer encoder-decoder structure. On the other hand, we confirmed that the AQT scored lower than the baseline in the state-space complex video game task. We also confirmed that the introduction of the TTQ was effective in some of the stage-type game tasks.



(a) Encoder-attention.: Example visualization of a scene in which a ball is hit back with a paddle. The visualization examples are arranged horizontally in time order.



(b) Decoder-attention.: The caption below the attention indicates the name and Q-value of the action. The caption at the bottom of the raw image indicates the action selected by the agent in the current frame. “Fire” in Breakout behaves the same as “Noop”.

FIGURE 7. Visualization example of attention in Breakout.Arrows show the direction of travel of the ball.

3) VISUAL EXPLANATIONS USING ATTENTION MAP

We visualized the attention using the AQT model for Breakout and the AQT+TTQ model for Seaquest, as shown in figs. 7 and 8, respectively. The model with the highest score in each game was used to visualize the attention (i.e., the AQT model for Breakout and the AQT+TTQ model for Seaquest). In the following, we explain the agent’s decision-making using the attention shown in these figures.

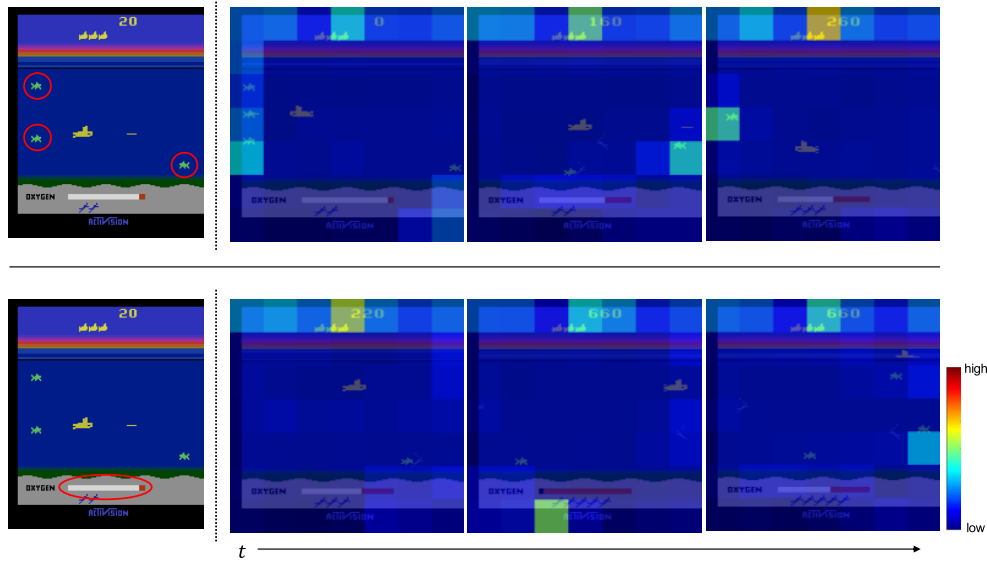
4) BREAKOUT

Breakout is a game in which the player hits the ball back with the paddle to destroy the upper blocks. There are four actions of the agent (=paddle): “Noop”, “Fire”, “Left”, and “Right”.

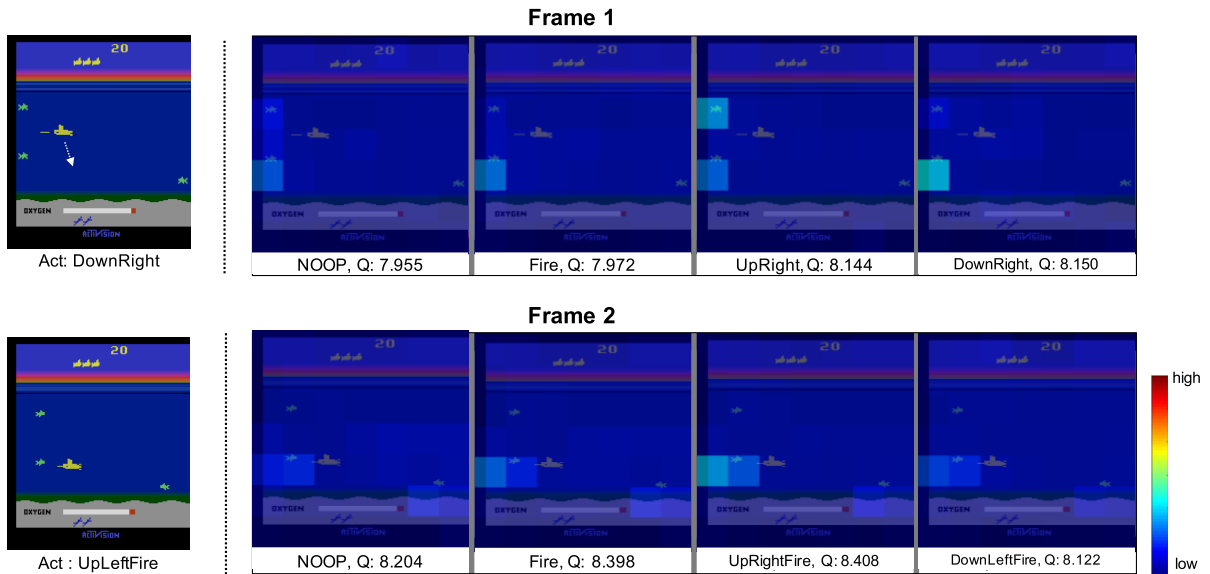
Encoder attention (state-value). An example of encoder attention visualization of a scene in Breakout where a ball is hit back is shown in Fig. 7a. Here, the agent is gazing at the paddle at the moment the ball is hit back and at the area with the fewest number of blocks. In Breakout, the score is obtained by destroying the block. Therefore, an important

state for the agent is the state in which the ball is hit back with the paddle. In addition, when the agent makes a hole in a block and hits the ball back into it, the ball bounces on the top of the block and destroys many blocks. Based on the above, we believe that the agent correctly recognizes that the paddle and the ball are important objects at the moment of hitting the ball back, and that a hole in the block is important for obtaining a high score.

Decoder attention (advantage). An example of visualization of decoder attention in Breakout is shown in Fig. 7b. At Frame 1, the agent is gazing at the right side of the paddle for “Right” and at the left side of the paddle for “Left”. In Breakout, the agent’s action “Right” moves the paddle to the right and “Left” moves it to the left. Therefore, the agent is considered to be aware of the destination for each action. At Frame 2, the agent gazes at the paddle in all actions, and the higher the action value, the more strongly the agent gazes at the paddle. Frame 2 is a scene where the ball is approaching the paddle, so the agent needs to control the paddle to match the ball. Based on the above, we can assume



(a) Encoder-attention.: The top is an example of visualization for fish (red circles) in several frames. The bottom is an example of visualization for an oxygen gauge (red circle) in a scene where oxygen has been depleted and replenished. The visualization examples are arranged horizontally in time order.



(b) Decoder-attention.: The caption below the attention indicates the name and Q-value of the action. The caption at the bottom of the raw image indicates the action selected by the agent in the current frame. The white arrow indicates the direction of the submarine. The total number of actions taken by Seaquest agents is 18, but 4 actions are picked up and shown here.

FIGURE 8. Visualization example of attention in Seaquest.

that the more correctly the agent recognizes the destination of the paddle due to the action, and the more accurately the agent recognizes the paddle's destination and the ball's position, the better the agent can select the appropriate action.

5) SEAQUEST

Seaquest is a game in which the player must destroy enemies and fish while rescuing divers. The agent (submarine) has 18 actions in total: 17 combinations of “Fire”, “Up”, “Down”, “Left”, and “Right”, and an additional “Noop”.

Encoder-attention (state-value). An example of encoder-attention visualization focusing on fish and oxygen gauges in Seaquest is shown in Fig. 8a. At the top of the figure, the agent is gazing at the fish in all frames, and from the bottom, the agent is gazing at the oxygen gauge only for the frames in which it runs out of oxygen. In Seaquest, the score is obtained by destroying Fish. Also, the game is over when all the oxygen runs out, so the agent needs to replenish their oxygen supply at regular intervals. Based on the above, we can assume that the agent recognizes Fish as

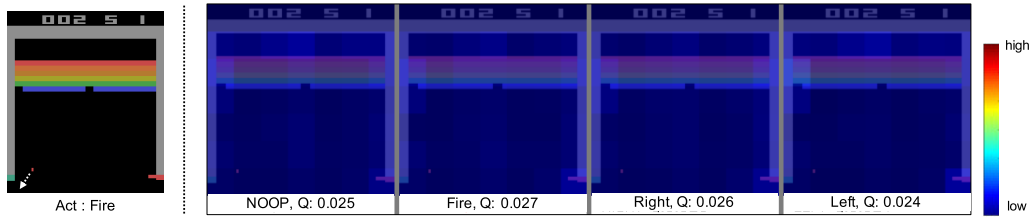


FIGURE 9. An example of attention visualization in a control failure scene in Breakout. This agent is an AQT model at training step 1.0×10^7 .

an important object in Seaquest and correctly recognize the need to replenish oxygen.

Decoder-attention (advantage). An example of visualization of decoder-attention in Seaquest is shown in fig. 8b. From Frame 1, the agent is gazing at the upper-left Fish in “UpRight” and at the lower-left Fish in “DownRight”. In Seaquest, the agent’s action “UpRight” moves the submarine to the upper right and “DownRight” moves it to the lower right. Hence, the direction of movement of the submarine due to the agent’s actions coincides with the position of Fish. Therefore, we can conclude that the agent is gazing at the Fish located at the destination. In Frame 2, which has been moved to the lower right after “DownRight” was selected in Frame 1, “UpLeftFire” was selected, which includes an attack. In other words, in Frame 2, the agent is attacking the Fish it had gazed at in Frame 1. Based on the above, we consider that the agent recognizes Fish as a target to be destroyed and correctly identifies the most efficient target to attack for each action.

6) DISCUSSION

By using the output of the encoder as input to the value branch, the encoder calculates attention on state-values. The state-value represents the expected value of the return in the current state. Therefore, encoder attention indicates objects that are important for obtaining a score across all frames. By using the output of the decoder as input to the advantage branch, the decoder computes attention on the agent’s actions. In our method, the decoder uses action queries to calculate attention for each action, rather than for all actions. The advantage is the action value minus the state-value, which is the value of action alone. Therefore, the decoder attention indicates for each query (i.e., for each action) the objects or regions that are affected if that action were selected.

7) LIMITATIONS

In the above, we showed an example of attention visualization in a scene where the gazing area of the agent model is clearly obtained. Then, we focus on the scene in which the agent model of Breakout misses the ball. In other words, we focus on the scene in which the agent fails to control the ball and discuss the limitations of AQT. Fig. 9 shows an example of attention visualization in a control failure scene in Breakout. Here, the agent does not hit the ball back, because

the ball falls under the paddle and the selected action is also “Fire”. In this control failure scene, there is no difference in Q-values for each action, and there is no clear gazing region for attention. These results are not informative enough to reveal the reason for the control failure of the agent model. This suggests that although AQT can analyze agent decision making in detail, there are limitations in its application. In particular, attention visualization may be less likely to provide meaningful information at a stage when the agent has not learned appropriate behavior or in situations of high environmental uncertainty. Since various factors contribute to control failures in agent models, we consider it necessary to establish an XRL method that takes into account not only the agent but also the entire DRL framework.

B. ROBOT CONTROL TASK

In this section, we describe an experiment using a robot’s autonomous movement task in an indoor environment. In this experiment, we focus on an autonomous movement task in an indoor environment using the Human Support Robot (HSR) [51]. HSR is a life support robot developed by Toyota Motor Corporation, which aims to support a wide range of people’s lives. Since HSR operates in the same living space as people, reliability of HSR’s control is a critical factor. Therefore, we evaluate the autonomous movement performance of the robot agent by AQT in a simulation environment and analyze the robot agent’s decision-making by visualizing its attention. We also apply the robot agent learned in the simulation environment to the real environment and evaluate the effectiveness of visual feedback using an AR interface.

To use the simulation environment, we created an indoor CG environment with multiple obstacles, including a walking human and furniture, using Unity [52]. When applying a learned robot agent in a simulation environment to a real-world environment, performance degradation due to domain gaps in input information is an issue. In this experiment, segmentation images are used as an intermediate representation to reduce the domain gap. For this reason, the texture in the simulation was assigned a different color for each object, as in fig. 10 (Objects such as sofas and desks are red, floors are blue, and walls are green). When the robot operates in the real world, a semantic segmentation model is used to obtain segmentation images from RGB images

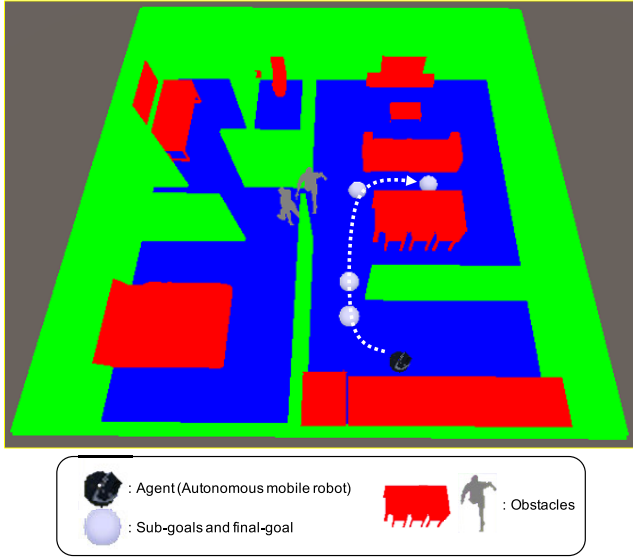


FIGURE 10. Simulated indoor environment created in unity.

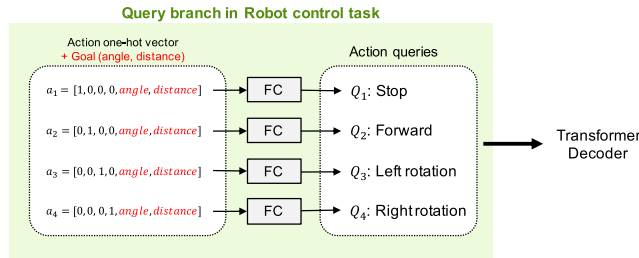


FIGURE 11. Query branch with goal information in robot control tasks.

acquired from the robot's camera. This segmentation image is used as input to the agent model to reduce domain gaps caused by differences in texture. The semantic segmentation model used in this experiment is SegNet [53], which has been well trained on real-world data.

The goal of the autonomous movement task is for the robot to efficiently reach the final goal while avoiding obstacles. In this simulation environment, the start and final goal positions are randomly set for each episode, and several sub-goals in between are automatically generated. The robot agent's actions are defined as: stop, move forward, rotate left, and rotate right. The following rewards were designed to solve this task. Each reward is shown in eq.4.

$$\text{reward} = \begin{cases} 10 & (\text{reach the goal}) \\ -0.01 & (\text{not getting closer to goal}) \\ -3 & (\text{collision with something}) \end{cases} \quad (4)$$

If the distance between the robot agent and the goal position is less than 0.25m, the robot is given a goal reaching reward. If the robot agent is not close to the goal, a small negative reward is given to encourage the robot to reach the goal. If the robot agent collides with some object, it is given a penalty for collisions reward to encourage it to avoid the obstacle. The

TABLE 1. Task accomplishment rate during 100 episodes.

Agent model	DQN	AQT (Ours)
Task accomplishment rate [%]	77	83

total reward in this experiment is the sum of these three types of rewards.

As the baseline model for AQT in this experiment, we employ a representative value-based DRL algorithm, Deep Q-Network (DQN) [2]. The compared methods are the baseline model "DQN", and our method "AQT". The end of training for each model is set at 1.0×10^6 steps. The end condition of the episode was set to 100 steps.

The following three evaluation methods were used.

- Performance evaluation of robot autonomous movement in indoor environments
- Visual explanations via attention map
- Visual feedback to users using AR

1) DETAILS OF ROBOT AGENT MODEL

Since DQN is a model that outputs Q-values, the action branch of AQT in this experiment is the structure described in sec. III-D. Since the robot agent has four actions, the action branch outputs four Q-values.

In this experiment, the purpose is to move the robot agent autonomously to the goal. Therefore, it is necessary to provide the agent model with segmentation images obtained from the robot's camera images and goal information as additional information. Thus, we added the goal information to the action query of the AQT model in order to promote action selection with consideration of the goal information. The action query with goal information in this experiment is shown in fig.11. Since the robot agent performs four actions (stop, forward, left rotation, right rotation), we define a one-hot vector for each action. These one-hot vectors are combined with the goal information that represents the relative angle and distance between the robot and the goal. The angle information is represented as a continuous value between 0 and 1 which is proportional to the goal's angle relative to the robot. When the goal is just behind the robot, the angle value is 0, and the angle value increases as the goal moves clockwise. The distance information is set to one-tenth of the distance between the robot and the goal, to maintain consistency with the scale of other information. Each combined vector is converted to an embedded vector by the FC layer and used as a query (action query) in the decoder.

2) PERFORMANCE EVALUATION OF ROBOT AUTONOMOUS MOVEMENT IN INDOOR ENVIRONMENTS

To confirm the autonomous control performance of AQT in the robot control task, we evaluated its performance in terms of the task accomplishment rate during 100 episodes. Here, a task is considered accomplished if the robot agent reaches the final goal within 100 steps and the number

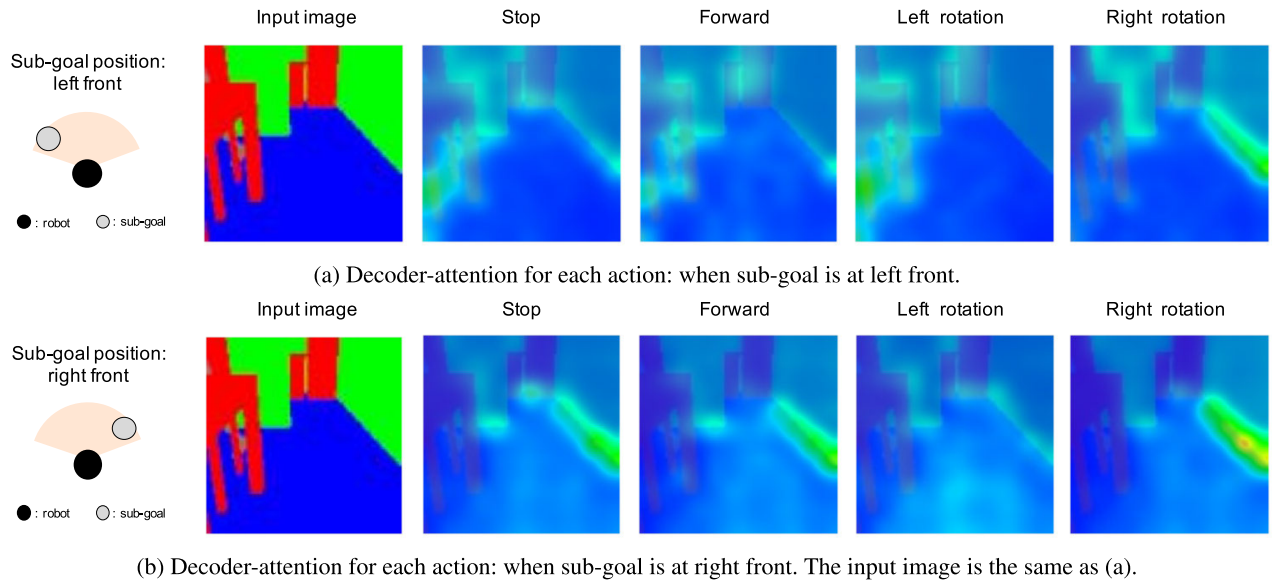


FIGURE 12. Visualization of decoder-attention in robot autonomous movement tasks.

of collisions with other objects is less than 6 times. This task accomplishment was measured for 100 episodes and expressed as a percentage. In other words, the higher the task accomplishment rate means that the agent has achieved a high level of autonomous control performance. The DQN model structure, used as the baseline model, consists of three convolutional layers and three FC layers. As in the AQT model, the input is a segmentation image obtained from the robot's camera image. The goal information is converted to an embedding vector in the FC layer, which is concatenated with the convolutional layer outputs.

As shown in table 1, AQT has a higher task accomplishment rate than DQN by more than 6%, reaching over 80%. This confirms that by introducing the Transformer structure, the AQT model achieves high control performance in the robot's autonomous movement task.

3) VISUAL EXPLANATION USING ATTENTION MAP

In this section, we describe the effectiveness of AQT for robot agent decision making in an autonomous movement task. We evaluate the effectiveness of AQT by visualizing the decoder-attention of a robot agent (AQT model) in a simulation environment. The AQT decoder in this experiment was input a one-hot vector of the action and the goal position information as a query. Then, we confirmed whether the decoder-attention for each action changes appropriately by changing the goal position. Fig. 12 (a), (b) show the attention when the goal position is 2 meters ahead on the left side and 45 degrees on the right side, respectively. Here, both (a) and (b) use the same input image, and only the goal position is changed.

From fig. 12 (a), we can see that the agent gazes at the furniture on the left side when selecting "Left rotation", and

gazes at the wall on the right side when selecting "Right rotation". On the other hand, "Stop" is not an action that affects the surroundings in the current frame, so the agent gazes at a wide area. Therefore, it is considered that the robot agent gazes at the area or object that is affected by each action of the robot agent. From fig. 12 (a) and (b), we confirm that when the goal position changes from left forward to right forward, the gazing target of the robot agent also changes. In the "Stop" and "Forward", the target of the robot agent's gaze moves from the furniture on the left side to the wall on the right side. The "Left rotation" reduces the focus on the furniture on the left side, and the "Right rotation" significantly increases the gazing at the wall on the right side. Thus, the robot agent gazes at objects closer to the goal position, and gazes at objects more intensely when the selected action is closer to the gazing target.

According to these results, decoder-attention correctly indicates the robot agent's gazing area for each action and goal position, and visualization of attention can be useful for users to understand the robot's decision-making.

4) VISUAL FEEDBACK TO USERS USING AR

In this section, we describe the effectiveness of a visual feedback method for users using an AR interface. In this experiment, we investigate the effectiveness of visual feedback by decoder-attention visualization of the AQT model using an AR interface, based on the user's understanding of the robot agent's behavior. The two investigation methods are as follows.

- Correct response rate to robot behavior prediction questions with and without visual feedback using AR
- A subjective questionnaire investigation on understanding robot behavior

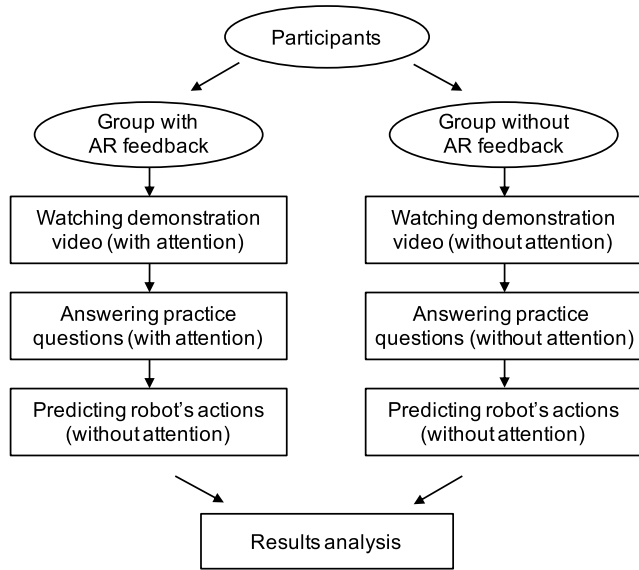


FIGURE 13. Flowchart for the investigation of robot behavior prediction.

We investigated the participants' capability to predict the robot's behavior by dividing 33 participants into two groups, one with and the other without visual feedback of attention by AR. In this investigation, the simulation environment shown in fig. 5 (a) was used to conduct an exhaustive evaluation of the robot's behavior. Fig. 13 shows the flowchart of the investigation, and the procedure is described in detail below.

- 1) Participants watch a demonstration video of a robot performing an autonomous movement task. The demo video is provided differently for each participant group. The group with AR visual feedback was provided the demo video with attention visualization using AR, while the other group was shown the normal demo video.
- 2) Participants answered 10 questions to predict the robot's behavior from the provided static images to promote their understanding of the robot's behavior. The static images were visualized in different ways for each group, the same way as the demonstration images provided in 1). In other words, the group with AR visual feedback could understand the robot's behavior including the robot's gazing area in the 3D space.
- 3) Participants answered 10 questions to predict the robot's behavior based on normal still images (no visual feedback from the attention). Then, the percentage of correct answers to these questions was calculated to investigate the effectiveness of visual feedback with AR.

The average percentage of correct answers for all participants is shown in table 2. We can see that the group with AR visual feedback increased the average correct answer rate of all participants by 7% compared to the group without AR visual feedback. This result indicates that AR visual feedback can be effectively useful for users in predicting the robot's behavior. In other words, the visual feedback via AR is

TABLE 2. Users' prediction of robot actions.

Participants' group	Without AR feedback	With AR feedback
Average accuracy [%]	31.8	38.8

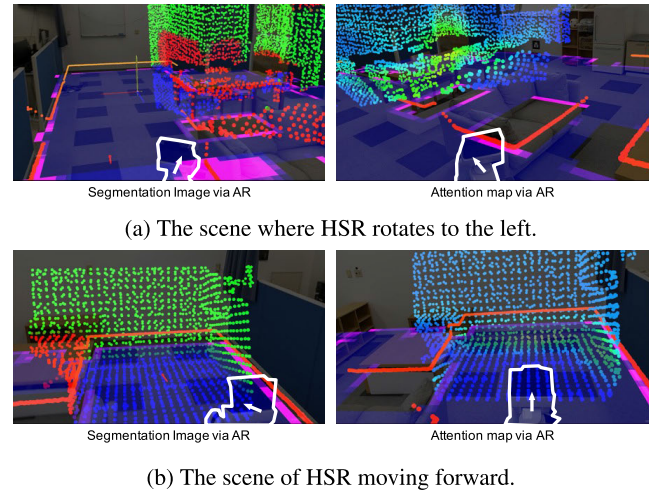


FIGURE 14. Example of visual feedback of attention using AR device in the real world. The area surrounded by a white frame contains the HSR, and the white arrow indicates the direction in which the HSR is moving.

effective in deepening the user's understanding of the robot's behavior.

Subjective Understanding of Robot Behavior: In order to confirm whether visual feedback using AR-based attention visualization improves the user's subjective understanding of robot behavior, we conducted a questionnaire investigation with users. In order to confirm the effectiveness of AR-based visual feedback in the real world, we conducted a real-world experiment as shown in fig. 5 (b).

The investigation method is described below. The 23 participants were asked to watch two different videos of the robot operating autonomously in the real world. One is a video taken by a fixed-point camera (observed via the naked-eye), and the other is a video in which the attention is visualized in the real world using an AR interface (observed via the AR). An example of the visualization of attention by the AR interface in the real world is shown in fig. 14. After watching the video, we conducted a subjective questionnaire to evaluate the understanding of the robot's behavior. In this questionnaire, participants were asked to rate their understanding on a four-point scale: "Extremely well", "Moderately well", "Slightly well", and "Not at all well". The videos were shot in four different scenes in the real world using HoloLens2 [54], an AR device developed by Microsoft Corporation. The results of the understanding questionnaire for each video are shown in table 3. The percentage of participants who answered "Extremely well" and "Moderately well" increased in the AR observation compared to the naked-eye observation. These results indicate that AR-based visual

TABLE 3. Questionnaire results of understanding robot behavior. Each number shows the percentage of respondents, and bold numbers indicate a high percentage.

Level of understanding	Percentage of answers [%]	
	obs. via naked-eyes	obs. via AR [%]
Extremely well	14.1	30.6
Moderately well	41.3	45.8
Slightly well	22.8	18.1
Not at all well	21.7	5.6

feedback of attention is effective in promoting the user's subjective understanding of the robot's behavior.

VI. CONCLUSION

In this paper, we propose Action Q-Transformer (AQT), an XRL method based on a Transformer encoder-decoder structure that uses action information as a query, with the aim of improving the interpretability of agents' decision-making. The AQT decoder uses an action query that expresses action information as a query to obtain unique attention for each action. This allows the agent to obtain the attention for all possible actions it could select, and enables a detailed analysis of the agent model's decision-making. In addition, we propose a new visual feedback method for users that visualizes the agent's attention using augmented reality (AR) as a framework for users to understand the agent intuitively and immediately, aiming to promote users' understanding of robots operating in real space. Our method visualizes the attention of a robotic agent's decision-making in real space using AR, allowing users to easily understand the reasons for the agent's decision to select an action via an AR device.

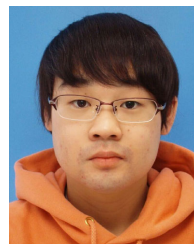
In our experiments using the Atari 2600, we confirmed that the encoder can obtain state-valued attention and the decoder can obtain unique attention for each action by using the DRL algorithm Rainbow as a base. From this experiment, we confirmed that decoder-attention has different distinct areas of attention corresponding to each behavior. By visualizing these different attentions that represent the agent's decision-making, we were able to effectively analyze the agent's decision-making in the video game strategy task. Furthermore, we proposed a Target Trained Q-network (TTQ) as a technique for solving the Transformer scaling rule in the AQT, and demonstrated that it achieves higher scores than the baseline. These experiments were also conducted using the Human Support Robot (HSR) in an indoor environment. We confirmed that the robot agent's interpretability was improved by obtaining attention for each action, and that its control performance was improved by evaluating its task accomplishment rate. We also conducted an evaluation experiment of the AR-based visual feedback method using HSR, and investigated the correct answer rate for user predictions of the robot's behavior and the user's comprehension questionnaire in a real environment. The experiment results confirmed that AR-based visual feedback of attention is effective in promoting user understanding of the robot's

behavior, indicating that it has the potential to improve human-robot coexistence. Future work includes performance improvement of the agent model and experiments with real-world applications other than video game strategy and robotics.

REFERENCES

- [1] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, May 1992.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, L. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2020.
- [5] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, Jan. 2020, pp. 213–229.
- [6] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lucic, and C. Schmid, "ViViT: A video vision transformer," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 6816–6826.
- [7] L. Chen, K. Lü, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2021, pp. 15084–15097.
- [8] J. Wang, P. Karanasou, P. Wei, E. Gatti, D. M. Plasencia, and D. Kanoulas, "Long-short decision transformer: Bridging global and local dependencies for generalized decision-making," in *Proc. Int. Conf. Learn. Represent.*, 2025.
- [9] J. Kim, S. Lee, W. Kim, and Y. Sung, "Decision ConvFormer: Local filtering in MetaFormer is sufficient for decision making," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2024.
- [10] Q. Zheng, A. Zhang, and A. Grover, "Online decision transformer," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2022, pp. 27042–27059.
- [11] K. Shao, Z. Tang, Y. Zhu, N. Li, and D. Zhao, "A survey of deep reinforcement learning in video games," 2019, *arXiv:1912.10944*.
- [12] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen, "Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection," *Int. J. Robot. Res.*, vol. 37, nos. 4–5, pp. 421–436, Apr. 2018.
- [13] X. Liang, X. Du, G. Wang, and Z. Han, "A deep reinforcement learning network for traffic light cycle control," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1243–1253, Feb. 2019.
- [14] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. A. Sallab, S. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 4909–4926, Jun. 2022.
- [15] Z. Liu, C. Yao, H. Yu, and T. Wu, "Deep reinforcement learning with its application for lung cancer detection in medical Internet of Things," *Future Gener. Comput. Syst.*, vol. 97, pp. 1–9, Aug. 2019.
- [16] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving, "Fine-tuning language models from human preferences," 2019, *arXiv:1909.08593*.
- [17] A. Mott, D. Zoran, M. Chrzanowski, D. Wierstra, and D. J. Rezende, "Towards interpretable reinforcement learning using attention augmented agents," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, Jan. 2019, pp. 12329–12338.
- [18] E. M. Kenny, M. Tucker, and J. Shah, "Towards interpretable deep reinforcement learning with human-friendly prototypes," in *Proc. Int. Conf. Learn. Represent.*, 2023.
- [19] S. Greydanus, A. Koul, J. Dodge, and A. Fern, "Visualizing and understanding Atari agents," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2017, pp. 1792–1801.

- [20] H. V. Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-Learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, Feb. 2016, pp. 2094–2100.
- [21] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," 2015, *arXiv:1511.05952*.
- [22] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, Jun. 2016, pp. 1995–2003.
- [23] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2017, pp. 449–458.
- [24] M. Fortunato, M. G. Azar, B. Piot, J. Menick, I. Osband, A. Graves, V. Mnih, R. Munos, D. Hassabis, O. Pietquin, C. Blundell, and S. Legg, "Noisy networks for exploration," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2017.
- [25] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 3215–3222.
- [26] A. P. Badia, B. Piot, S. Kapturowski, P. Sprechmann, O. Vitvitskiy, Z. D. Guo, and C. Blundell, "Agent57: Outperforming the Atari human benchmark," in *Proc. Int. Conf. Mach. Learn.*, vol. 1, Jul. 2020, pp. 507–517.
- [27] S. Kapturowski, V. Campos, R. Jiang, N. Rakićević, H. van Hasselt, C. Blundell, and A. Puigdomènech Badia, "Human-level Atari 200x faster," 2022, *arXiv:2209.07550*.
- [28] K.-H. Lee, O. Nachum, M. Yang, L. Lee, D. Freeman, W. Xu, S. Guadarrama, I. Fischer, E. B. Jang, H. Michalewski, and I. Mordatch, "Multi-game decision transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, Jan. 2022, pp. 27921–27936.
- [29] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. S. Novikov, G. Barth-Maron, M. Giménez, Y. Sulsky, J. Kay, J. T. Springenberg, T. Eccles, J. Bruce, A. Razavi, A. Edwards, N. Heess, Y. Chen, R. Hadsell, O. Vinyals, M. Bordbar, and N. de Freitas, "A generalist agent," *Trans. Mach. Learn. Res.*, vol. 2022, Jan. 2022.
- [30] H. Zheng, L. Shen, Y. Luo, T. Liu, J. Shen, and D. Tao, "Decomposed prompt decision transformer for efficient unseen task generalization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 1–11.
- [31] V. Micheli, E. Alonso, and F. Fleuret, "Transformers are sample-efficient world models," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2023.
- [32] V. Micheli, E. Alonso, and F. Fleuret, "Efficient world models with context-aware tokenization," in *Proc. Int. Conf. Mach. Learn.*, Jun. 2024.
- [33] M. Wen, J. G. Kuba, R. Lin, W. Zhang, Y. Wen, J. Wang, and Y. Yang, "Multi-agent reinforcement learning is a sequence modeling problem," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2022, pp. 16509–16521.
- [34] S. Hu, L. Shen, Y. Zhang, and D. Tao, "Learning multi-agent communication from graph modeling perspective," in *Proc. Int. Conf. Learn. Represent.*, May 2024.
- [35] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 618–626.
- [36] L. Weikamp, E. van der Pol, and Z. Akata, "Visual rationalizations in deep reinforcement learning for Atari games," in *Proc. BNAIC*, Jan. 2019, pp. 151–165.
- [37] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [38] A. Manchin, E. Abbasnejad, and A. van den Hengel, "Reinforcement learning with attention that works: A self-supervised approach," in *Proc. Int. Conf. Neural Inf. Process.*, Jan. 2019, pp. 223–230.
- [39] R. Zhang, Z. Liu, L. Zhang, J. A. Whritner, K. S. Müller, M. M. Hayhoe, and D. H. Ballard, "AGIL: Learning attention from human for visuomotor tasks," in *Proc. Eur. Conf. Comput. Vis.*, Jan. 2018, pp. 692–707.
- [40] C. Rupprecht, C. Ibrahim, and C. J. Pal, "Finding and visualizing weaknesses of deep reinforcement learning agents," in *Proc. Int. Conf. Learn. Represent.*, 2020.
- [41] W. Shi, G. Huang, S. Song, Z. Wang, T. Lin, and C. Wu, "Self-supervised discovering of interpretable features for reinforcement learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 5, pp. 2712–2724, May 2022.
- [42] Y. Bakkemoen and H. Langseth, "ASAP: Attention-based state space abstraction for policy summarization," in *Proc. Asian Conf. Mach. Learn.*, 2024, pp. 137–152.
- [43] O. Mahjoub, R. J. de Kock, S. Singh, W. Khelifi, A. Vall, K. A. Tessler, R. Gorsane, and A. Pretorius, "Efficiently quantifying individual agent importance in cooperative Marl," in *Proc. AAAI Conf. Artif. Intell. Explainable AI Approaches Deep Reinforcement Learn. (XAI4DRL) Workshop*, 2024.
- [44] K. K. Miskiw and P. Staudt, "Explainable deep reinforcement learning for multi-agent electricity market simulations," in *Proc. 20th Int. Conf. Eur. Energy Market (EEM)*, Jun. 2024, pp. 1–9.
- [45] R. Suzuki, A. Karim, T. Xia, H. Hedayati, and N. Marquardt, "Augmented reality and robotics: A survey and taxonomy for AR-enhanced human-robot interaction and robotic interfaces," in *Proc. CHI Conf. Human Factors Comput. Syst.*, Apr. 2022, pp. 1–33.
- [46] Z. Makhataeva, A. Zhakatayev, and H. A. Varol, "Safety Aura visualization for variable impedance actuated robots," in *Proc. IEEE/SICE Int. Symp. Syst. Integr. (SII)*, Jan. 2019, pp. 805–810.
- [47] H. Hedayati, M. Walker, and D. Szafir, "Improving collocated robot teleoperation with augmented reality," in *Proc. 13th ACM/IEEE Int. Conf. Human Robot Interact. (HRI)*, Mar. 2018, pp. 78–86.
- [48] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
- [49] J. T. Springenberg, A. Abdolmaleki, J. Zhang, O. Groth, M. Bloesch, T. Lampe, P. Brakel, S. Bechtle, S. Kapturowski, R. Hafner, N. Heess, and M. Riedmiller, "Offline actor-critic reinforcement learning scales to large models," in *Proc. Int. Conf. Mach. Learn.*, Feb. 2024.
- [50] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI gym," 2016, *arXiv:1606.01540*.
- [51] T. Yamamoto, K. Terada, A. Ochiai, F. Saito, Y. Asahara, and K. Murase, "Development of human support robot as the research platform of a domestic mobile manipulator," *ROBOMECH J.*, vol. 6, no. 1, pp. 1–15, Dec. 2019.
- [52] A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar, and D. Lange, "Unity: A general platform for intelligent agents," 2018, *arXiv:1809.02627*.
- [53] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder-decoder architecture for image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.
- [54] *Microsoft Hololens2 Docs*. Accessed: Mar. 22, 2025. [Online]. Available: <https://www.microsoft.com/it-it/hololens>



HIDENORI ITAYA (Graduate Student Member, IEEE) received the B.E. degree in computer science and the M.S. degree from Chubu University, Japan, in 2019 and 2021, respectively, where he is currently pursuing the Ph.D. degree. He has been with Denso Wave Inc., since 2021.



WANTAO YIN received the B.E. degree in mechanical design manufacture and automation from Jiaxing University, China, in 2020, and the M.S. degree in robot science and technology from Chubu University, Japan, in 2024.



TSUBASA HIRAKAWA (Member, IEEE) received the Ph.D. degree in computer science from Hiroshima University, Japan, in 2017. From 2017 to 2019, he was a Researcher Fellow with Chubu University, Japan, where he has been a specially appointed Associate Professor with Chubu Institute for Advanced Studies, Chubu University, since 2019. He has also been a Lecturer with the Department of Computer Science, Chubu University, since 2021. He was a Fellowship of Japan Society for the Promotion of Science, from 2014 to 2017. He was a Visiting Researcher with ESIEE Paris, France, in 2014 and 2015.



TAKAYOSHI YAMASHITA (Member, IEEE) received the Ph.D. degree in computer science from Chubu University, Japan, in 2011. He was with Omron Corporation, from 2002 to 2014, and was a Lecturer with the Department of Computer Science, Chubu University, from 2014 to 2017, where he was also an Associate Professor, from 2017 to 2021. He has been a Professor with Chubu University, since 2021. His current research interests include object detection, object tracking, human activity understanding, pattern recognition, and machine learning. He is a member of the IEICE and the IPSJ.



HIRONOBU FUJIYOSHI (Member, IEEE) received the Ph.D. degree in electrical engineering from Chubu University, Japan, in 1997. From 1997 to 2000, he was a Postdoctoral Fellow with the Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, USA, working on the DARPA video surveillance and monitoring (VSAM) effort and the humanoid vision project for the HONDA humanoid robot. He is currently a Professor with the Department of Robotics, Chubu University, Japan. From 2005 to 2006, he was a Visiting Researcher with the Robotics Institute, Carnegie Mellon University. His research interests include computer vision, video understanding, and pattern recognition. He is a member of the IEICE and the IPSJ.



KOMEI SUGIURA (Member, IEEE) received the B.E. degree in electrical and electronic engineering and the M.S. and Ph.D. degrees in informatics from Kyoto University, in 2002, 2004, and 2007, respectively. He is currently a Professor with Keio University, Japan. From 2006 to 2008, he was a Research Fellow with Japan Society for the Promotion of Science. From 2006 to 2009, he was also with the ATR Spoken Language Communication Research Laboratories. From 2008 to 2020, he was a Senior Researcher with the National Institute of Information and Communications Technology, Japan, before joining Keio University, in 2020. His research interests include multimodal language understanding, service robots, machine learning, spoken dialogue systems, cloud robotics, imitation learning, and recommender systems.

...